DSAA 4011 · FRIDAY 13 NOVEMBER 2026 · 15:00–15:50 · E1-150

From Pixels to Latents

What can an image encoder keep, and what can a generator recover?

Yuwen Huang · Laboratory 10

20 minutes of explanation; 30 minutes for your work

Finish the application together, then explore the student experiments.

Minutes	Activity
0–5	Recall encoder, decoder and latent space
5–15	Compression, lost detail and reconstruction
15–20	Attention costs and latent diffusion
20–45	Student experiments, alone or in pairs
45–50	Student questions and discussion

SO **One image story, three small experiments.**

NEXT What must we remember from the Gaussian example?

Compress an existing image; rebuild it from the code

Encoder: image → smaller code. Decoder: code → reconstructed image. First understand reconstruction; generation comes later.

Component	Yesterday	Image application
Observation x	Noisy number 1.6	Visible image
Hidden z	Hidden clean number	Latent code
Encoder	Distribution given x	Code distribution given pixels
Decoder	Observation likelihood	Image distribution given a code

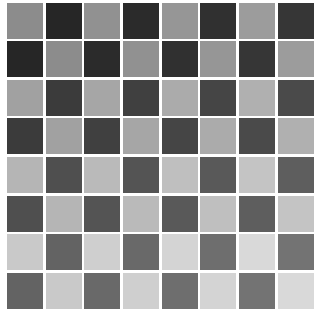
SO **A smaller code is useful only if the decoder can recover the needed details.**

NEXT What does a small image actually contain?

Start with one picture containing visible detail

Look at the alternating light and dark pixels. We will compress this same picture throughout the lab.

8 × 8 original: 64 pixels



A light–dark pattern lives inside every 2 × 2 block.

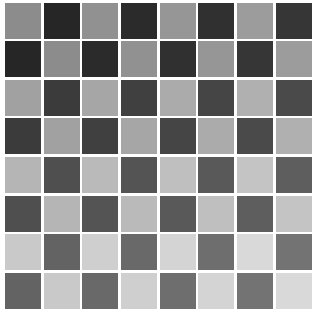
SO **The detail to watch is the light–dark alternation inside each 2 × 2 block.**

NEXT What remains if four pixels are replaced by their average?

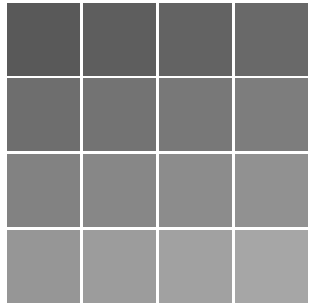
Four pixel values become one block average

First block: (0.55 + 0.15 + 0.15 + 0.55) / 4 = 0.35. Apply this rule to every block.

Original image



4 × 4 block means: 16 values



Each code value averages four pixels; opposite details cancel.

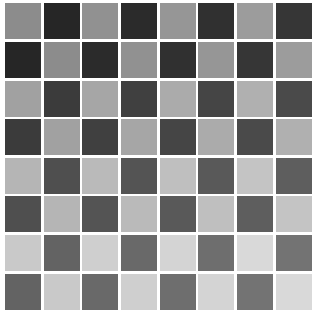
SO **The code keeps 16 averages instead of 64 original pixels.**

NEXT Can repeating each average recreate the original detail?

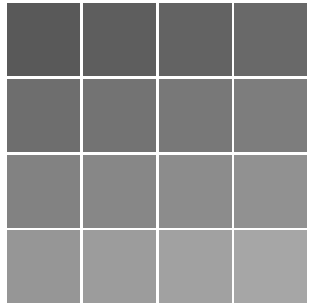
Repeating an average cannot recreate lost detail

Left: original. Middle: block averages. Right: repeat each average into four pixels. Compare the checkerboard detail.

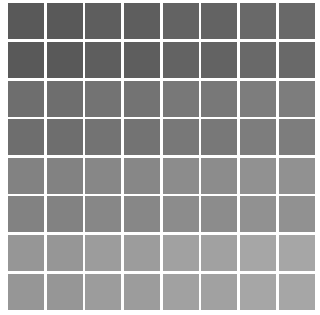
Original



Mean-only code



Mean-only reconstruction



The code gets smaller, but within-block detail disappears.

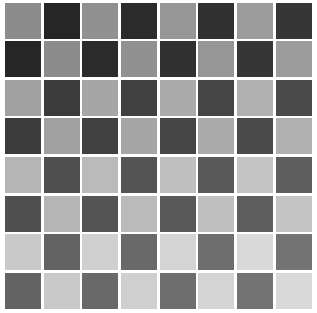
SO **The reconstruction preserves coarse brightness but loses the alternating detail.**

NEXT How can we prove the original detail cannot be recovered from these means?

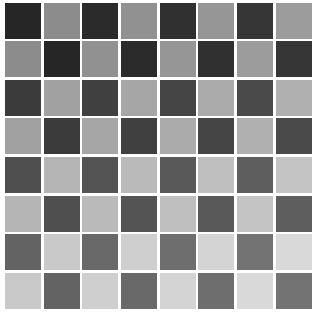
Identical codes cannot identify different originals

Pictures A and B reverse the light–dark detail, but every block still has the same average.

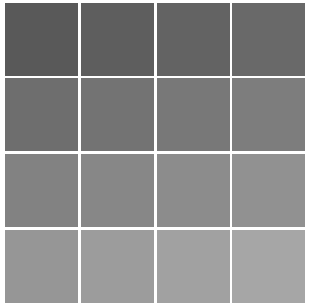
Original A



Original B



Both give the same mean code



Different pictures, identical code: the decoder cannot identify which original was input.

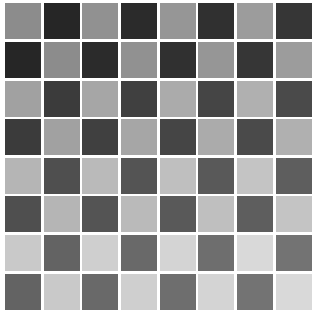
SO **The mean-only code cannot tell A from B; neither can a decoder receiving only that code.**

NEXT What additional value could distinguish these two patterns?

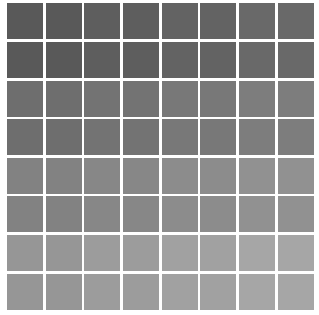
Keep one extra value describing the contrast

For the first toy block, store mean 0.35 and contrast 0.20: bright pixels are 0.35 + 0.20; dark pixels are 0.35 − 0.20.

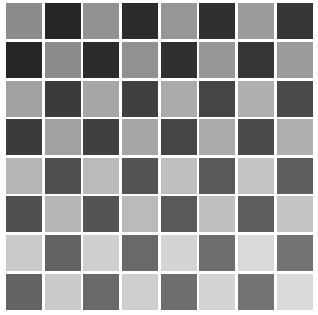
Original



Means alone



Means + contrast: exact toy decode



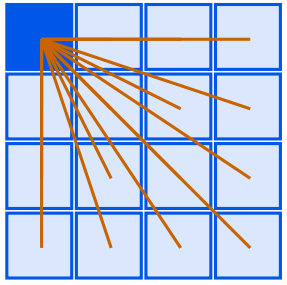
One extra coefficient per block restores this constrained checkerboard detail.

SO **Mean plus contrast restores our specially constructed pattern; arbitrary blocks can require more information.**

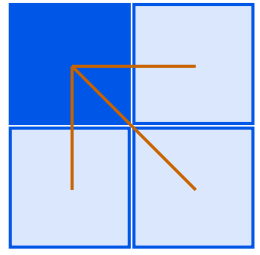
NEXT What computational saving is left after making a smaller grid?

A smaller grid requires fewer attention comparisons

One query cell compares itself with all cells. Repeat this for every query: N positions produce N × N comparisons.



one blue query looks at all 16 positions



one blue query looks at all 4 positions

Repeat for every query position: N queries, each comparing N positions.

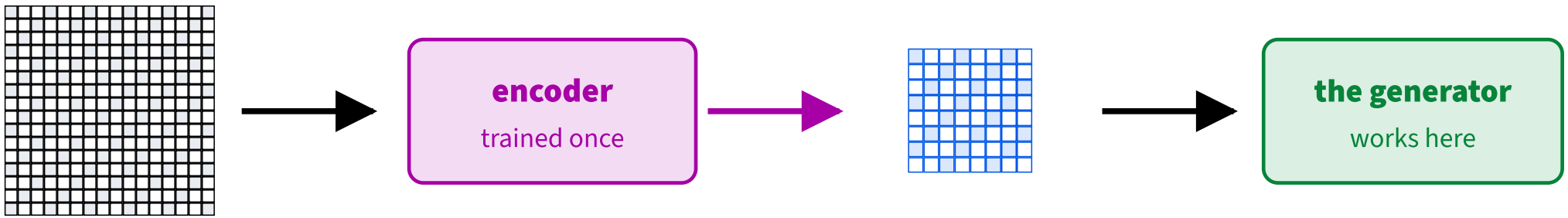
Small illustration	Positions N	Full-attention comparisons N × N
4 × 4 grid	16	16 × 16 = 256
2 × 2 grid	4	4 × 4 = 16

SO **Reducing positions from 16 to 4 reduces full-attention pairs from 256 to 16.**

NEXT How can an image generator use a smaller representation?

Generate the small code, then decode the image

The image encoder learns a useful small representation from training images. A separate generator learns to produce codes in that space.



the slow part of the system never sees a pixel: it sees the encoder's output

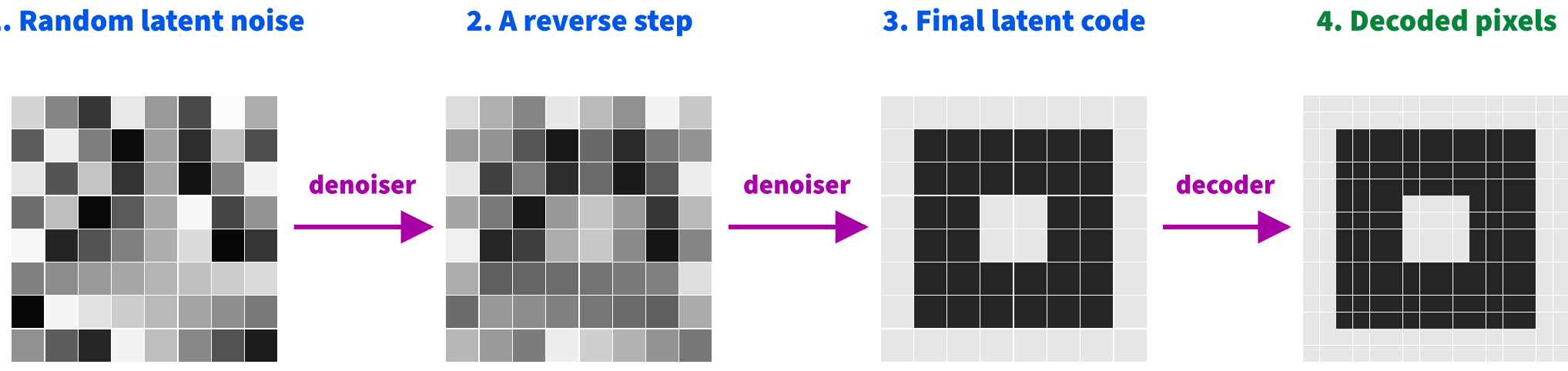
and a decoder turns the result back into a picture

SO **At generation time, produce a new latent code and decode it; no existing image needs to be encoded.**

NEXT How does diffusion produce a new code from noise?

The denoiser changes the code; the decoder renders it

Follow the four panels: random noise → a less noisy latent → final latent → decoded pixels.



The same trained denoiser is reused across noise levels; decoding is a separate operation.

Schematic grids illustrate the stages; these are not outputs of a trained diffusion model.

SO **Repeated denoiser updates generate the code; decoding is the final, separate operation.**

NEXT Now investigate information loss and noisy codes yourself.

Your turn: run, compare, explain

Start with Experiment 1. Use the remaining time for Experiments 2 and 3; work alone or with a partner.

Run	Explain
python lab10.py compress	Why can different pictures share one code?
python lab10.py attention	Do more channels mean more positions?
python lab10.py noise	Why is a decoder not automatically a denoiser?

SO **Open the generated SVG pictures and use them to explain your result.**

NEXT Bring one result or question to the final discussion.

Keep the code useful; measure the right costs

We finished the application, then tested information loss, representation size and latent noise.

Question	Check
What was lost?	Compare original images and shared codes
What became cheaper?	Count entries, positions and pairs separately
What is learned?	Distinguish encoder, decoder and denoiser

SO **A smaller representation buys efficiency only if it retains what the task needs.**

NEXT Use the student kit after class; diffusion training comes next week.

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Predict before compressing the picture

The encoder will replace each 2 × 2 block by its average.

Will the code preserve an alternating light–dark pattern?

No: opposite details can cancel | Yes: every detail survives | Only if we draw more samples

Answer. Averaging discards within-block contrast. A later decoder cannot identify which discarded detail was present.

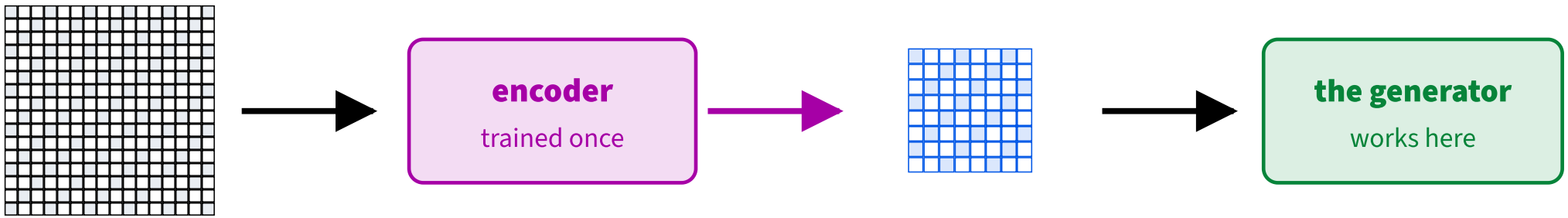
SO **Check your prediction before running Experiment 1.**

NEXT What does the averaged code contain?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

The hidden representation can make generation cheaper

Transfer the idea from **hidden variable** choices to hidden image representations.



the slow part of the system never sees a pixel: it sees the encoder's output

and a decoder turns the result back into a picture

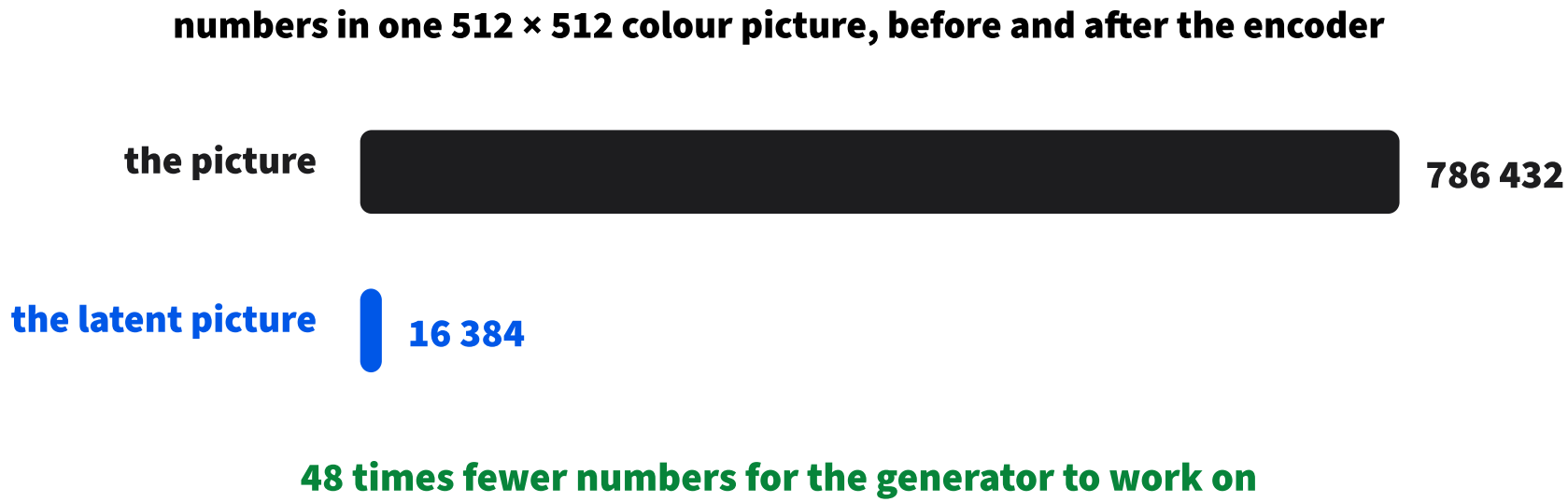
SO **An encoder and decoder connect pixels to a smaller latent space.**

NEXT Count the entries in one specific image and latent representation.

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Count the image entries before comparing costs

Count the entries in one specific image and **latent representation**.



SO **This design reduces the number of entries by a factor of 48.**

NEXT A smaller grid changes the number of positions an attention layer compares.

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Keep image details while regularising the hidden code

An image encoder produces a smaller hidden code. Its decoder must use that code to rebuild the original image's details.

Training requirement	In plain words
Reconstruction quality	Keep enough image information for the decoder
Prior penalty $\mathrm{KL}(q(z \mid x) \| p(z))$	Keep each guessed code distribution near the prior
Weight the two requirements	Decide how strongly to regularise while preserving useful details

> A simplified weighted objective: $\text{reconstruction reward} - \beta\,\mathrm{KL}(q(z \mid x) \| p(z))$. Here β sets the penalty weight.

If every image produced the same code distribution, the code would not distinguish their details. A weaker penalty may help retain information; it is not a guarantee. Image systems can also use perceptual and adversarial losses.

SO **Image autoencoders balance useful reconstruction and latent regularisation. A modified loss need not be the original lower bound $\mathcal{L}$.**

NEXT Where else do we reuse a learned function to infer hidden information?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Two numbers do not describe every 2 × 2 block

Four arbitrary pixel values have more freedom than our two-channel toy code.

Block type	Are mean and checkerboard contrast enough?
Our constrained toy blocks	Yes: the construction has exactly these two components
Arbitrary 2 × 2 block	No: other detail patterns can be missing

SO **Exact reconstruction is a property of the toy family, not a general guarantee.**

NEXT How does this connect to the prior penalty?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

A code must balance detail and regularisation

The mean-only toy illustrates discarded information. A variational encoder also pays for departing from its prior.

Quantity	Meaning
Reconstruction quality	Can the decoder recover useful details?
Kullback–Leibler (KL)(q(z | x) || p(z))	How far the code distribution moves from the prior
Penalty weight beta	How strongly training discourages that departure

SO **Compression loss and prior regularisation are related design pressures, but are not the same operation.**

NEXT What can we safely conclude from a weak prior penalty?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Experiment 2: count work before claiming speed

Run: python lab10.py attention. It prints position counts and full-attention pair counts.

Compare	Prediction
8 × 8 pixels versus 4 × 4 code	How many spatial positions in each?
One versus two code channels	Do spatial positions change?
Full attention	Square the position count, not the channel count

SO **Counts explain a possible computational saving; they are not measured end-to-end speedups.**

NEXT What does the extra channel change?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Channels and positions are different counts

A second value at each code cell adds channels but does not add spatial positions.

Representation	Entries	Positions	Position pairs
8 × 8 × 1	64	64	4096
4 × 4 × 1	16	16	256
4 × 4 × 2	32	16	256

SO **Entries = positions × channels; full-attention pairs = positions squared.**

NEXT How do these counts appear in a real latent-image design?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Experiment 3: inspect a noisy latent code

Run: python lab10.py noise. Add seeded noise to the same two-channel code and decode it.

Change	Inspect
Noise level zero	Toy code reconstructs the toy image
Increase seeded latent noise	Decoded image error grows in this controlled construction
Interpretation	A learned denoiser is a missing component, not an automatic decoder property

SO **This illustrates how latent noise affects reconstruction; it does not train or simulate a diffusion sampler.**

NEXT Can decoding alone remove arbitrary latent noise?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

The toy explains a mechanism, not image quality

Our encoder is hand-written, our image family restricted, and our noise experiment has no learned denoiser.

Toy	Real application
Means and contrast coefficients	Learned multichannel latent representation
Hand-written reconstruction	Learned image decoder
Noise injection only	Trained multi-step denoiser
Exact constrained images	Lossy approximate image reconstruction

SO **Do not equate our block averages with a trained variational autoencoder.**

NEXT Which roles nevertheless transfer to real systems?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

The application uses yesterday's components

The same distinctions recur even when the hidden representation becomes a whole grid.

Lecture concept	Where it appears
Encoder q(z | x)	Infer a code distribution from an existing image
Decoder p(x | z)	Rebuild observations from latent values
Prior regularisation	Shape the code distribution during autoencoder training
Latent diffusion	Learn a generative process for latent representations

SO **Name the role before naming the architecture.**

NEXT Can you explain the entire pipeline in your own words?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Check: can the decoder recover discarded details?

Two original pictures produce exactly the same code. No other information reaches the decoder.

Can a deterministic decoder reconstruct both originals exactly?

No: identical input gives identical output | Yes: train the decoder longer | Yes: enlarge only the output grid

Answer. No. It can choose or invent a reconstruction, but cannot know which of the two originals produced that code.

SO **A downstream model cannot identify information the code does not distinguish.**

NEXT What should we measure when judging a latent system?

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Sources and scope

The image-generation connection follows the latent diffusion paper; the small block codec is our teaching construction.

Source	Use
Rombach et al., CVPR 2022	Latent diffusion and image autoencoders
Kingma and Welling, International Conference on Learning Representations (ICLR) 2014	Variational encoder and training objective
Lecture 10	Lower bound, inference gaps and sampling gradients

SO **The experiments are transparent mechanism checks, not reproductions of an image generator.**

NEXT Optional backup: calculate one block by hand.

OPTIONAL REFERENCE · NOT PART OF THE 20-MINUTE EXPLANATION

Backup: one block, two coefficients

For one toy block, pixels are mean plus or minus the same contrast.

Pixels	Mean	Contrast
[[0.8, 0.2], [0.2, 0.8]]	(0.8 + 0.2 + 0.2 + 0.8) / 4 = 0.5	(0.8 − 0.2 − 0.2 + 0.8) / 4 = 0.3

SO **Averaging gives the mean; alternating signs recover the contrast.**

NEXT Use this calculation to check the code.